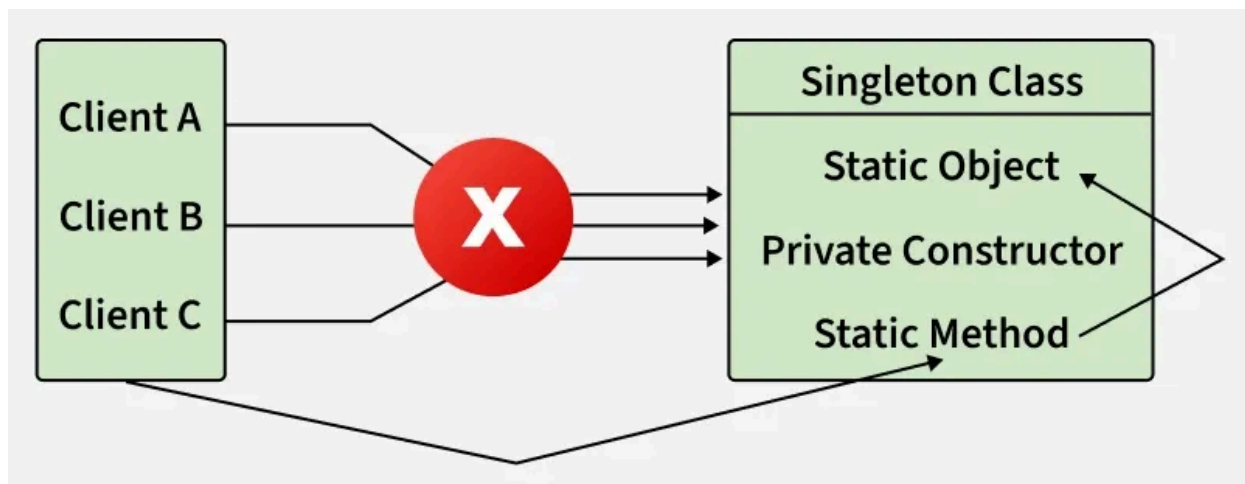


Singleton Method Design Pattern

Last Updated : 04 Dec, 2025

The Singleton Design Pattern ensures that a class has only one instance and provides a global access point to it. It is used when we want centralized control of resources, such as managing database connections, configuration settings or logging.



Real-World Applications of the Singleton Pattern

1. **Logging Systems** : Maintain a consistent logging mechanism across an application.
2. **Configuration Managers** : Centralize access to configuration settings.
3. **Database Connections** : Manage a single point of database access.
4. **Thread Pools** : Efficiently manage a pool of threads for concurrent tasks.
5. **Cache Managers, Print Spoolers (Single Printer Queue) and Runtime Environments** (`java.lang.Runtime` is a singleton)

Features of the Singleton Design Pattern

1. **Single Instance:** Ensures only one object of the class exists in the JVM.
2. **Global Access Point:** Provides a centralized way to access the instance.
3. **Lazy or Eager Initialization:** An Instance can be created at class load time (eager) or when first needed (lazy).
4. **Thread Safety:** Can be designed to work correctly in multithreaded environments.
5. **Resource Management:** Useful for managing shared resources like configurations, logging or database connections.
6. **Flexibility in Implementation:** Can be implemented using eager initialization, lazy initialization, double-checked locking or an inner static class.

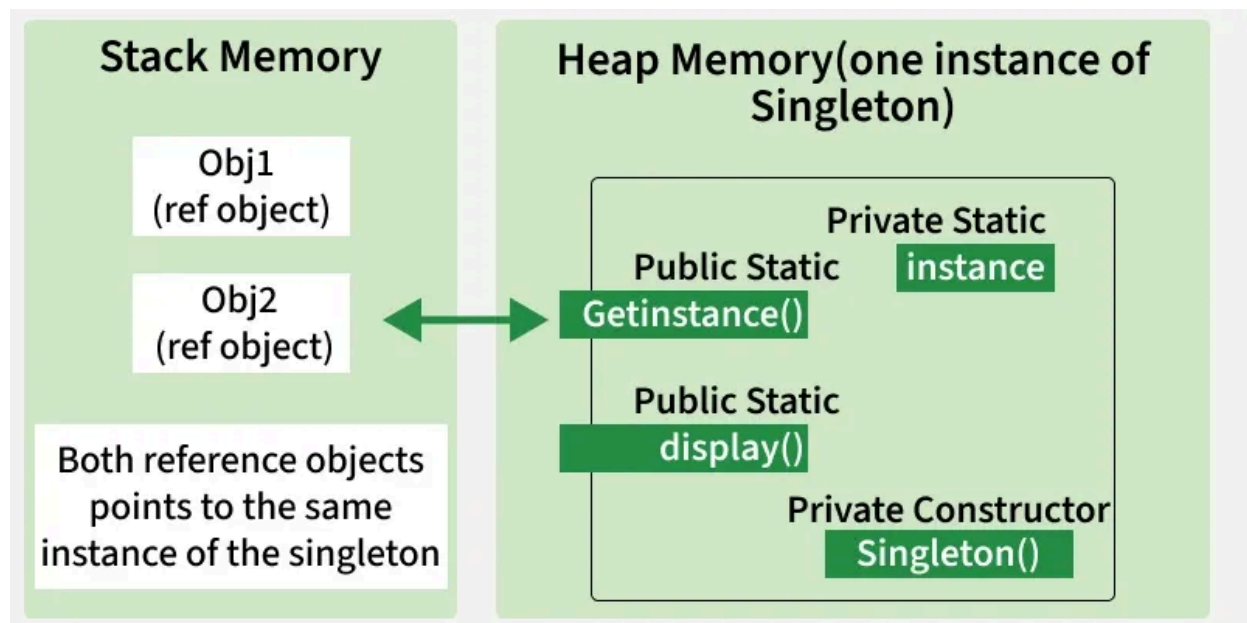
When would you like to use the Singleton Method Design Pattern

Use the Singleton method Design Pattern when:

- Consider using the Singleton pattern when you need to ensure that only one instance of a class exists in your application.
- If you think you might want to extend the class later, the Singleton pattern is a good choice. It allows for subclassing, so clients can work with the extended version without changing the original Singleton.
- It's important to use it judiciously. Overuse can lead to issues such as hidden dependencies and testing difficulties.

Key Components

Below are the main key components of Singleton Method Design Pattern:



Key Components

1. Static Member

The Singleton pattern or pattern Singleton employs a static member within the class. This static member ensures that memory is allocated only once, preserving the single instance of the Singleton class.

```
// Static member to hold the single instance
private static Singleton instance;
```



2. Private Constructor

The Singleton pattern or pattern singleton incorporates a private constructor, which serves as a barricade against external attempts to create instances of the Singleton class. This ensures that the class has control over its instantiation process.

```
// Private constructor to prevent external instantiation
class Singleton {

    // Making the constructor as Private
    private Singleton()
    {
        // Initialization code here
    }
}
```



```
}  
}
```

3. Static Factory Method

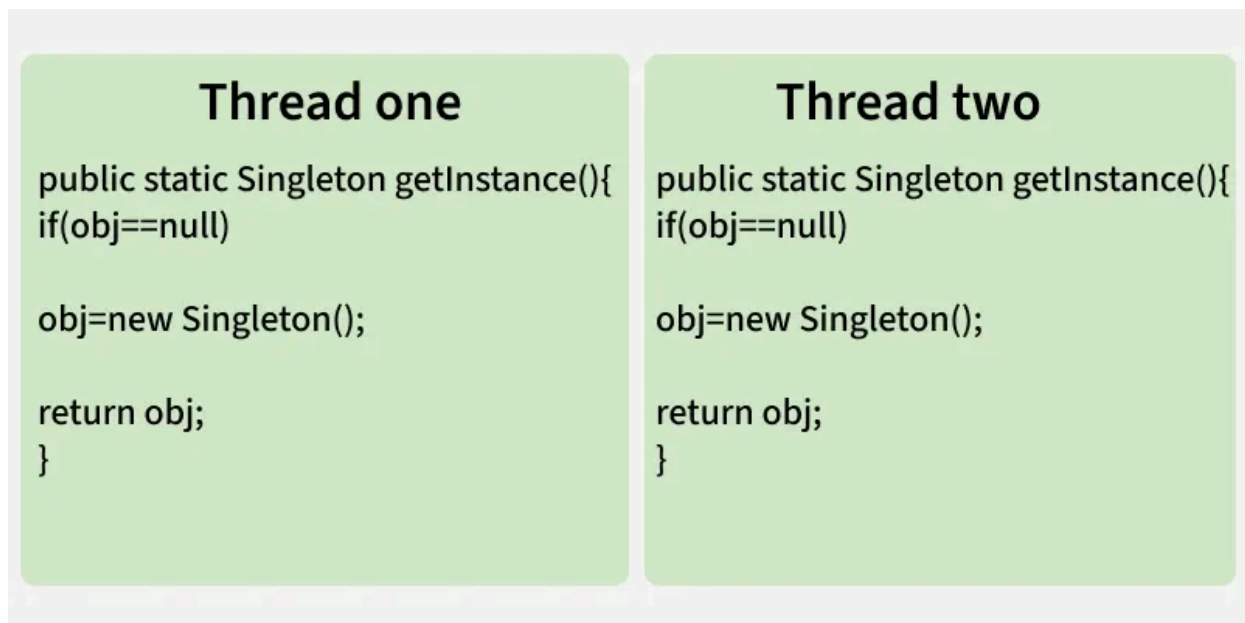
A crucial aspect of the Singleton pattern is the presence of a static factory method. This method acts as a gateway, providing a global point of access to the Singleton object. When someone requests an instance, this method either creates a new instance (if none exists) or returns the existing instance to the caller.

```
// Static factory method for global access  
public static Singleton getInstance()  
{  
    // Check if an instance exists  
    if (instance == null) {  
        // If no instance exists, create one  
        instance = new Singleton();  
    }  
    // Return the existing instance  
    return instance;  
}
```



Different Ways to Implement Singleton Method Design Pattern

Sometimes we need to have only one instance of our class for example a single DB connection shared by multiple objects as creating a separate DB connection for every object may be costly. Similarly, there can be a single configuration manager or error manager in an application that handles all problems instead of creating multiple managers.



Let's see various design options for implementing such a class. If you have a good handle on static class variables and access modifiers this should not be a difficult task.

1. Classic (Lazy Initialization)

In this method, class is initialized whether it is to be used or not. The main advantage of this method is its simplicity. You initiate the class at the time of class loading. Its drawback is that class is always initialized whether it is being used or not.

Example: Classical Java implementation of singleton design pattern

```
class Singleton {
    private static Singleton obj;

    // private constructor to force use of getInstance() to create Singleton
    object
    private Singleton() {}

    public static Singleton getInstance()
    {
        if (obj == null)
            obj = new Singleton();
        return obj;
    }
}
```

Here we have declared **getInstance()** static so that we can call it without instantiating the class. The first time **getInstance()** is called it creates a new singleton object and after that, it just returns the same object.

***Note:** Singleton obj is not created until we need it and call the `getInstance()` method. This is called lazy instantiation. The main problem with the above method is that it is not thread-safe. Consider the following execution sequence.*

This execution sequence creates two objects for the singleton. Therefore this classic implementation is not thread-safe.

2. Thread-Safe (Synchronized)

Make `getInstance()` synchronized to implement Singleton Method Design Pattern

Example: Thread Synchronized Java implementation of singleton design pattern

```
class Singleton {  
    private static Singleton obj;  
    private Singleton() {}  
  
    // Only one thread can execute this at a time  
    public static synchronized Singleton getInstance()  
    {  
        if (obj == null)  
            obj = new Singleton();  
        return obj;  
    }  
}
```

Here using synchronized makes sure that only one thread at a time can execute **getInstance()**. The main disadvantage of this method is that using synchronized every time while creating the singleton object is expensive and may decrease the performance of your program. However, if the performance of **getInstance()** is not critical for your application this method provides a clean and simple solution.

3. Eager Initialization (Static Block)

In this method, class is initialized only when it is required. It can save you from instantiating the class when you don't need it. Generally, lazy initialization is used when we create a singleton class.

Example: Static initializer based Java implementation of singleton design pattern

```
class Singleton {  
    private static Singleton obj = new Singleton();  
    private Singleton() {}  
  
    public static Singleton getInstance() { return obj; }  
}
```

Here we have created an instance of a singleton in a static initializer. JVM executes a static initializer when the class is loaded and hence this is guaranteed to be thread-safe. Use this method only when your singleton class is light and is used throughout the execution of your program.

4. Double-Checked Locking (Most Efficient)

Use “Double Checked Locking” to implement singleton design pattern

Example: Double Checked Locking based Java implementation of singleton design pattern

```
class Singleton {  
    private static volatile Singleton obj = null;  
    private Singleton() {}  
  
    public static Singleton getInstance()  
    {  
        if (obj == null) {  
            // To make thread safe  
            synchronized (Singleton.class)  
            {  
                // check again as multiple threads can reach  
                above step  
            }  
        }  
    }  
}
```

```

        if (obj == null)
            obj = new Singleton();
    }
}
return obj;
}
}

```

We have declared the obj volatile which ensures that multiple threads offer the obj variable correctly when it is being initialized to the Singleton instance. This method drastically reduces the overhead of calling the synchronized method every time.

5. Static Inner Class (Best Java-Specific Way)

In Java, a Singleton can be implemented using a static inner class.

- A class is loaded into memory only once by the JVM.
- An inner class is loaded only when it is referenced.
- Therefore, the Singleton instance is created lazily, only when the getInstance() method accesses the inner class.

Example: using class loading concep singleton design pattern

```

public class Singleton {

    private Singleton() {
        System.out.println("Instance created");
    }

    private static class SingletonInner{

        private static final Singleton INSTANCE=new Singleton();
    }
    public static Singleton getInstance()
    {
        return SingletonInner.INSTANCE;
    }
}

```

In the above code, we are having a private static inner class SingletonInner and having private field. Through, getInstance() method of

singleton class, we will access the field of inner class and due to being inner class, it will be loaded only one time at the time of accessing the INSTANCE field first time. And the INSTANCE is a static member due to which it will be initialized only once.

6. Enum Singleton

In Java, a Singleton can also be implemented using an enum, which is the simplest and safest approach. Enums are loaded by the JVM only once, and each enum constant is created exactly one time. Therefore, the Singleton instance is created safely when the enum is first accessed.

Example: Enum-based Singleton

```
public enum Singleton {  
    INSTANCE;  
  
    public void doSomething() {  
        System.out.println("Doing something...");  
    }  
}
```

Explanation:

In this implementation

- INSTANCE is the single allowed object of the enum.
- The JVM ensures that the enum is thread-safe, created only once, and cannot be instantiated again.
- It also automatically protects against serialization and reflection issues.
- When Singleton.INSTANCE is accessed for the first time, the enum is loaded and the instance is initialized only once.

Implementation of the singleton Design pattern

Example: The implementation of the singleton Design pattern is very simple and consists of a single class.

```
import java.io.*;
```

```
class Singleton {  
    // static class  
    private static Singleton instance;  
    private Singleton()  
    {  
        System.out.println("Singleton is Instantiated.");  
    }  
    public static Singleton getInstance()  
    {  
        if (instance == null)  
            instance = new Singleton();  
        return instance;  
    }  
    public static void doSomething()  
    {  
        System.out.println("Something is Done.");  
    }  
}  
  
class GFG {  
    public static void main(String[] args)  
    {  
        Singleton.getInstance().doSomething();  
    }  
}
```

Output

```
Singleton is Instantiated.  
Something is Done.
```

Singleton Method Design Pattern

[Visit Course](#)[Comment](#)**K** **kartik** [+ Follow](#)**102****Article Tags :**[Design Pattern](#)[System Design](#)

Explore

What is System Design

System Design Fundamentals

[Functional and Non Functional](#) [Life Cycle](#) [Design Patterns](#) [UML Diagrams](#) [System Design](#)[Sign In](#)

Databases in Designing Systems

High Level Design(HLD)

Low Level Design(LLD)

Design Patterns

Interview Guide for System Design

System Design Interview Questions & Answers



Corporate & Communications Address:

A-143, 7th Floor, Sovereign Corporate Tower, Sector- 136, Noida, Uttar Pradesh (201305)

Registered Address:

K 061, Tower K, Gulshan Vivante Apartment, Sector 137, Noida, Gautam Buddh Nagar, Uttar Pradesh, 201305



Company

- About Us
- Legal
- Privacy Policy
- Contact Us
- Advertise with us
- GFG Corporate Solution
- Campus Training Program

Tutorials

- Programming Languages
- DSA
- Web Technology
- AI, ML & Data Science
- DevOps
- CS Core Subjects
- Interview Preparation

Explore

- POTD
- Job-A-Thon
- Blogs
- Nation Skill Up

Courses

- ML and Data Science
- DSA and Placements
- Web Development
- Programming Languages
- DevOps & Cloud
- GATE
- Trending Technologies

Software and Tools

Videos

DSA

Python

Java

C++

Web Development

Data Science

CS Subjects

Preparation Corner

Interview Corner

Aptitude

Puzzles

GfG 160

System Design

@GeeksforGeeks, Sanchhaya Education Private Limited, All rights reserved